

MARTHANDAM COLLEGE OF ENGINEERING AND TECHNOLOGY

Kuttakuzhi

Veeyannoor Post

(Near Marthandam)

Kanyakumari District



RECORD NOTE BOOK

AL3452-OPERATING SYSTEMS LABORATORY

NAME:

REG NO:

MARTHANDAM COLLEGE OF ENGINEERING AND TECHNOLOGY

Kuttakuzhi

Veeyannoor Post

Kanyakumari District



Certified that this is the bonafide record of work done by

Mr. / Ms. of the Third semester in Artificial Intelligence and Data Science Department of this college, in the AL3452- Operating Systems Lab during 2024- '25 in partial fulfillment of the requirements of the B.Tech Degree course of the Anna University, Chennai.

Staff-in-charge

Head of the Dept.

University Reg. No

University Exam held in.

Internal Examiner

External Examiner

[illegible]

[illegible]

Aim:

To install Windows/Linux Operating System.

How to install Windows?**INTRODUCTION:**

Each version of Microsoft Windows is installed on a computer using similar steps. While there are steps in the installation process that differ between versions of Windows, the following general steps and guidelines help you install Windows on your computer. The steps below are for all recent versions of Windows, including [Windows 98](#), [Windows ME](#), [Windows 2000](#), [Windows XP](#), [Windows Vista](#), [Windows 7](#), [Windows 8](#), [Windows 10](#), and [Windows 11](#). These steps even work for earlier versions (e.g., Windows 95) as long as you use the disc version. The floppy diskette version is similar, but it requires additional steps.

CHECK HARDWARE COMPATIBILITY

Before installing or upgrading Windows on your computer, check the hardware in the computer to make sure it's compatible with that version of Windows. Microsoft provides a [Windows Compatible Products List](#) for checking if the hardware in your computer is compatible with the chosen version of Windows.

If one or more pieces of [hardware](#) is not compatible with the chosen Windows version, we recommend replacing that hardware with compatible hardware or purchasing a new computer. Having compatible hardware in your computer helps ensure the Windows install or upgrade process is successful.

GENUINE WINDOWS CD,DVD,OR USB THUMB DRIVE

First, you need a genuine copy of the Microsoft Windows [operating system](#) installation [CD,DVD](#), or [USB thumb drive](#). A genuine Windows [product key](#) is included with the installation disc, which is required to activate Windows after installation. If you have an [OEM](#) (original equipment manufacturer) computer, the Windows product key is often on the back or side of the computer.

If you have an OEM computer (e.g., [Acer](#), [Dell](#), [HP](#), etc.), the computer will not have a genuine Windows CD, DVD, or USB thumb drive. Instead, you would reinstall Windows and the software using a [hidden partition](#) or a set of restore discs. The steps mentioned on this page would still work, but you'd need a copy of Windows. You can borrow a friend's Windows disc, as long as it's the same version of Windows that came with the computer and have a [product key](#).

HOW TO ENTER AND EXIT THE BIOS OR CMOS SETUP

Every computer provides a way to enter the [BIOS](#) or [CMOS](#) setup, which lets you configure some basic settings for your computer and its hardware components.

Below is a list of common methods for accessing and exiting your computer's BIOS or CMOS setup and recommendations if you're having trouble.

ACCESS BIOS OR CMOS ON NEWER COMPUTERS:

Computers manufactured in the last few years allow you to enter the BIOS or CMOS setup by pressing one of the five keys listed below during the [boot](#) process.

F1
F2
F10
Delete or Del
Esc

HOWDOICHANGEANDSAVECHANGES IN CMOS SETUP

Once in CMOS setup, the method for changing the settings often depends on the BIOS manufacturer. You may use the [arrow keys](#) and the Enter key to select categories and change their values. Some manufacturers may have you press the [Page up](#) and [Page down](#) keys to change the values. If you're trying to change the clock, speed, or other settings and don't have the option available, the mother board doesn't support it. If you believe it should be supported, you may need a BIOS update.

HOWDOISAVE THECHANGES:

If any changes are made, you need to save those changes, which are usually done by pressing the F10 key on the keyboard. If F10 doesn't work, look at the bottom or top of the screen for the key that's used to [save](#) the settings.

ACCESSBIOS ORCMOS ON OLDERCOMPUTERS:

Unlike today's computers, older computers (before 1995) had numerous methods of entering the BIOS setup. Below is a listing of key sequences to press as the computer boots to enter the BIOS setup.

Ctrl+Alt+Esc
Ctrl+Alt+Insert
Ctrl+Alt+Enter
Ctrl+Alt+S
PageUp
PageDown

AcerBIOS:

If your Acer computer cannot boot or you want to restore the BIOS to its original settings, press and hold theF10 as you turn on the computer. While holding F10, two beeps should be heard to indicate the settings are restored.

AMIBIOS:

Older AMI BIOS could be restored to bootable settings by pressing and holding ,Insert as the computer is booting.

BIOS or CMOS diskettes:

Early [486,386](#), and [286](#) computers required a floppy disk to enter the BIOS setup. These diskettes maybe called ICU, BBU, or SCU disks. Because these diskettes are unique to your computer manufacturer, you must obtain the diskettes from them. See the [computer manufacturers list](#) for contact information.

Access BIOS on early IBM computers:

Some early IBM computers require you to press and hold both [mouse](#) buttons as the computer boots to enter the BIOS setup.

HOW TO EXIT THE BIOS OR CMOS:

There are several ways to exit the BIOS or CMOS setup depending on the computer's type. The most common methods include the following.

Press the Esc key to exit without saving any changes.

Press the F10 or F12 key to save changes and exit.

Access the **Exit** or **Save & Exit** tab in setup and select the **Exit** or **Save and Exit** option.

If you have trouble exiting the BIOS or CMOS setup, you can try the following methods to fix the problem.

Press the F9 key to load default settings and press F10 to save and exit.

Access the **Exit** or **Save& Exit** tab in setup, select the **Load Optimized Defaults** option, select **Yes**, and press Enter. Any changes made are reverted, and the BIOS or CMOS is set back to default settings.

Turn off the computer(use only as a last resort and with caution).

INSTALLING OR UPGRADING WINDOWS:

To start the Windows install or upgrade process, you need to configure your computer to boot from a CD or DVD before booting to the hard drive. Changing the boot process forces the computer to look for the Windows installation disc before booting from the [hard drive](#).

1. Open the CMOS setup.
2. Change the computer's [boot order](#). Set the CD, DVD, or disc drive as the first boot device if you are trying to boot from a disc. Or, set the first boot device to your USB drive if you're trying to boot from a USB thumb drive. If the drive is not shown, keep the disc inserted and reboot the computer. With the disc in the drive, BIOS should recognize and include it in the list.
3. Save the settings change and exit BIOS.

Note: Once you have updated the boot order, you can begin the Windows installation process.

4. Place the Windows disc in the CD/DVD drive or USB thumb drive into the back of the computer.
5. Turn on or [restart the computer](#). As the computer starts up, it should detect the installation disc or drive and show a message similar to *Press any key to boot from CD*. Press any key on the keyboard to have the computer boot from the Windows disc or drive.
6. After the Windows install begins, there are several prompts that you need to answer. Select either **Yes** or the appropriate option to install Windows.
7. When asked which [partition](#) to install Windows onto, select the main partition, usually the C: drive or one labeled "Unallocated partition". If upgrading Windows, select the existing installation of Windows on the hard drive.
8. You may be asked if you want to erase all contents on the hard drive, then install Windows. We recommend you choose this option, as it also formats the hard drive to allow the Windows operating system to be installed.
9. The computer may need to restart several times during the Windows install process. The restarts are normal and if prompted to restart, select the **Yes** option.
10. When the install process is nearly complete, the Windows configuration option screens are shown. On these screens, you may be asked to select the time zone you live in, your preferred language, and the account's name you use to access Windows. Select the appropriate options and enter the appropriate information on each configuration screen.
11. The Windows install process is completed when the computer prompts you to log in or when it loads into Windows.

FINAL

WINDOWS AND COMPUTER CONFIGURATION:

After Windows is installed on the computer, you need to install the [drivers](#) and related software for the hardware in the computer. You can use the installation discs that came with the hardware or [download](#) the drivers from the hardware manufacturer's website. We strongly recommend you install the latest drivers for each piece of hardware.

To determine which hardware needs drivers to be installed, check the [Device Manager](#) and look for the [exclamation mark](#) "!" next to hardware devices. The exclamation point means drivers are needed for that device. After installing the necessary hardware device drivers, install any software programs on the computer that you want to use.

Finally, download and install any available Windows updates. Updating Windows improves the operating system's performance, the hardware in the computer, and the software programs you use. It can also improve security by fixing potential security holes and flaws in Windows.

LONG-TERM MAINTENANCE OF WINDOWS

Microsoft frequently releases new [updates](#) for Windows, so we recommend you check for and install available updates. Doing so helps keep Windows running better and keep your computer protected.

Also, periodically check for updated hardware device drivers from manufacturers' websites. Keeping hardware drivers updated helps the hardware devices in the computer run at peak performance and improves compatibility with other computer hardware and software.

How to install Linux?

If you want to install Linux, there are two ways to do that:

The first way is to download the Linux distribution you want and burn it into a DVD or USB stick and boot your machine with it and complete the installation process.

The second way is to install it virtually on a virtual machine like VirtualBox or VMware without touching your Windows or Mac system, so your Linux system will be contained in a window you can minimize and continue working on your real system.

For me, I prefer VirtualBox, it's free and runs very fast on my PC than VMware, and supports installing Windows, Linux, and Mac OS with all versions.

Let's choose any Linux distro and install it using both two ways.

I'm going to choose Linux mint; they call it the Mac OS of Linux. It is a good distro for personal use.

The version we are going to install is 18.1 "Serena" at the time of writing that article.

Go to this link and download it:

<https://www.linuxmint.com/download.php>

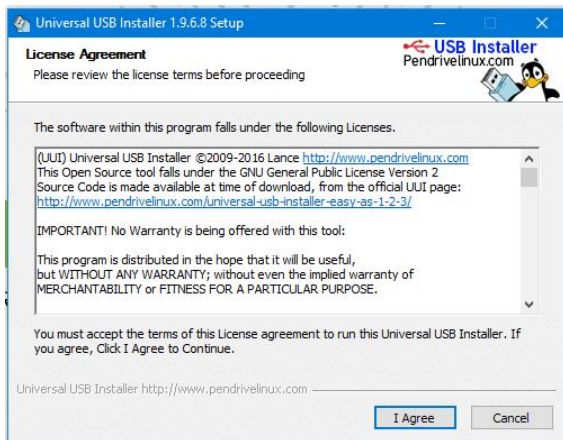
I prefer the Cinnamon desktop version; it is promising and elegant.

Once you download the ISO file, you will have to burn it on DVD or the easy way, copying it on a USB stick using a program called Universal USB installer, you can download this program from this link:

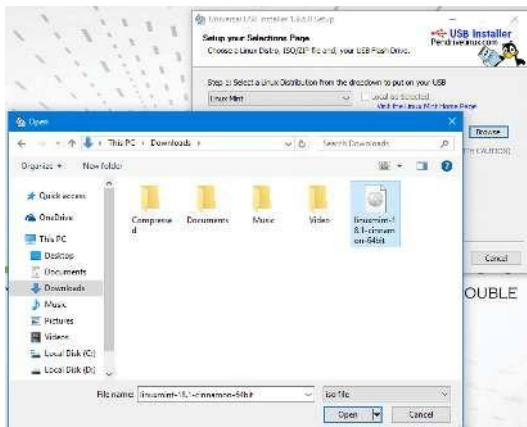
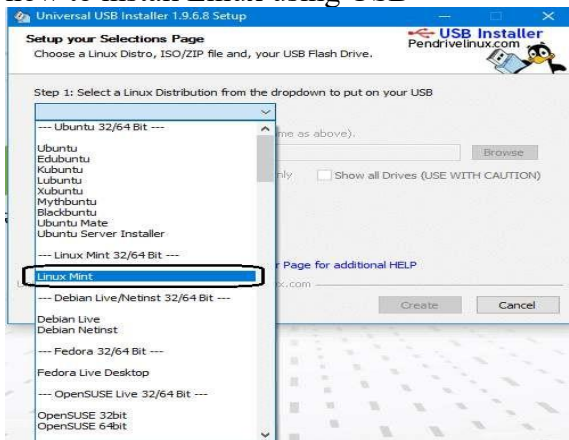
<https://www.pendrivelinux.com/universal-usb-installer-easy-as-1-2-3/>

After downloading the application, open it and choose from the list the distro you want to install; in our case, we will select Linux mint.

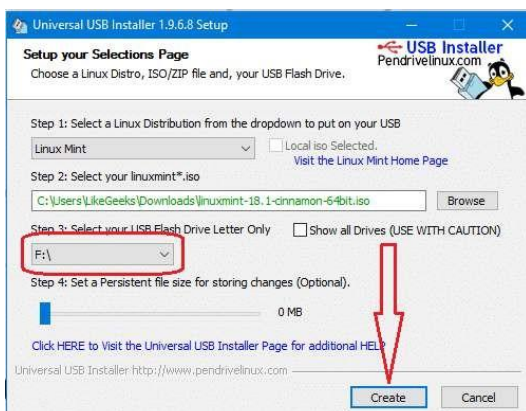
Make sure that you put your memory stick on the computer and click next and wait till the copying process is finished.



how to install Linux using USB



Now Finally we click create to create bootable USB



And now you can boot with this memory stick.

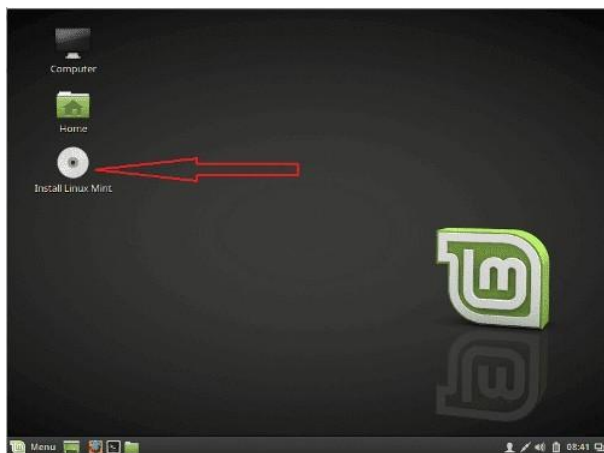
Then Restart your PC and go to BIOS settings and select boot options and make sure that the USB is the first option, then save your BIOS settings and reboot.



Then it will show the installation screen, press Enter, and it will load the live CD content.

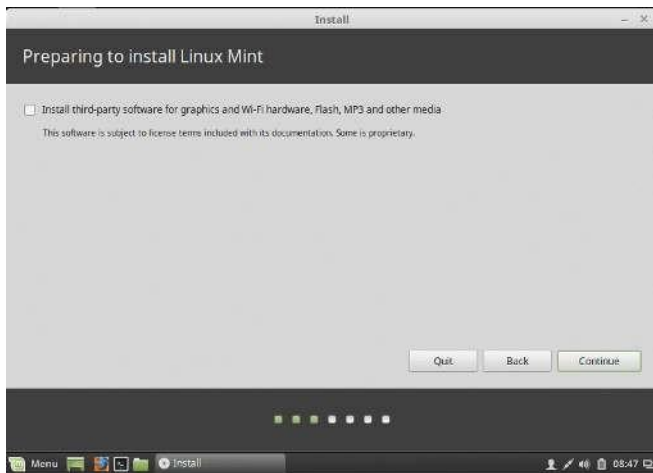
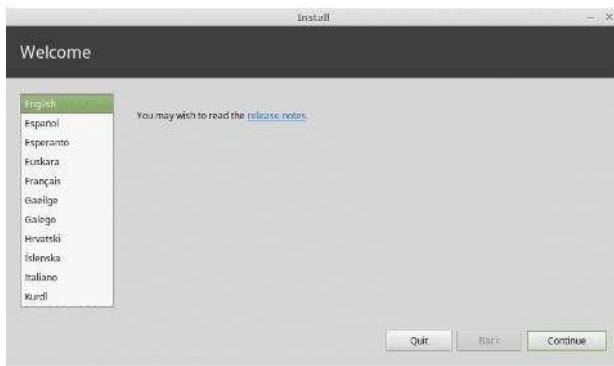


Now the desktop should appear like that:



Click **install Linux Mint**.

Then choose the language used for installation.



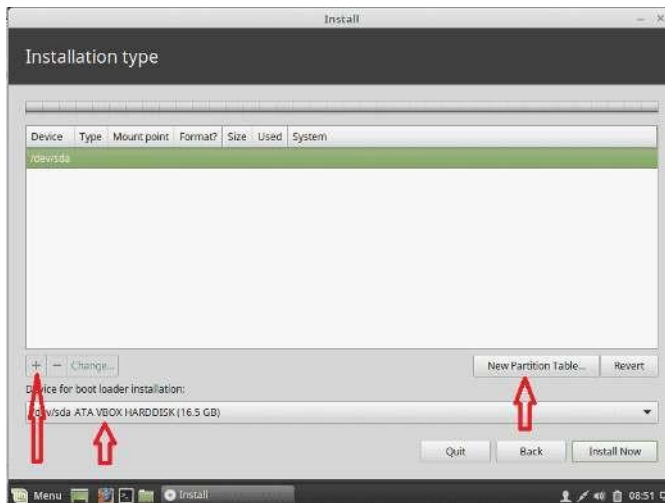
Then choose the installation type, and TAKE CARE if you are installing Linux on a disk that contains other operating systems, you MUST select the option called something else.



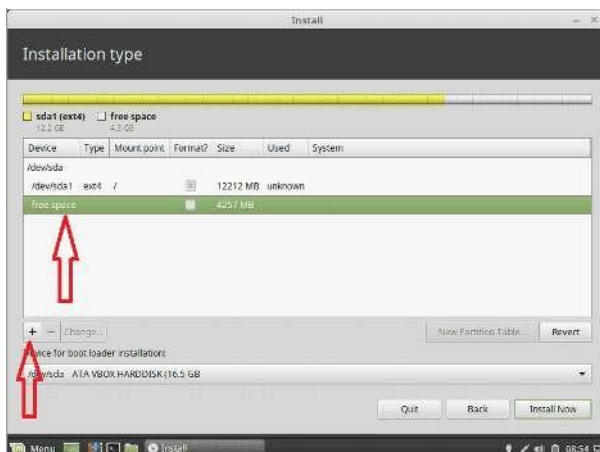
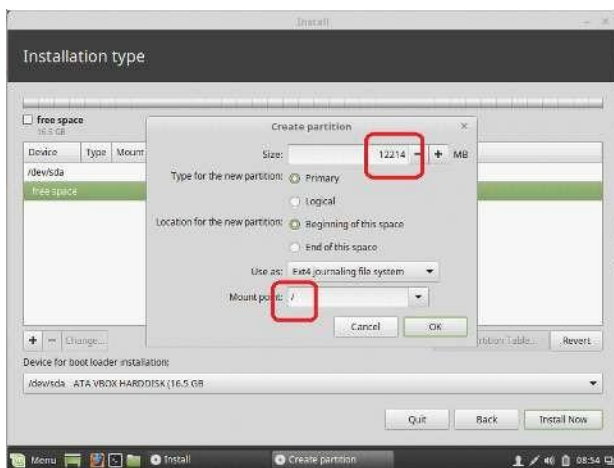
If you are installing it on a new disk, choose the option Erase disk.

Linux requires two partitions to work, the **root partition** and the **swap partition**.

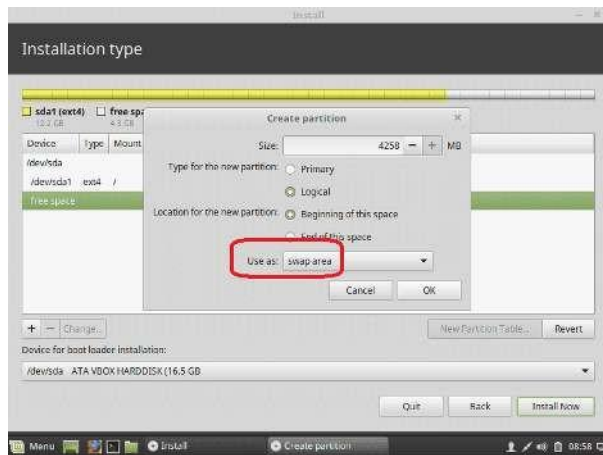
Now we will create them by clicking the plus button and choose about 12 GB or more as you need but not less than that for the root partition and choose mount type as / which stands for root and of course format will be **Ext4**.



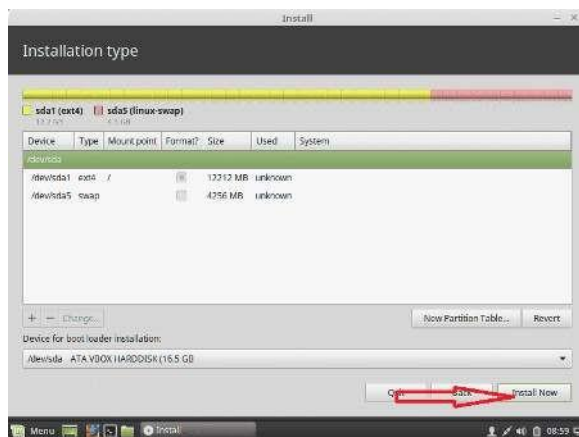
Now we create a swap partition, choose the remaining free space and click the plus button and choose the swap area as shown:



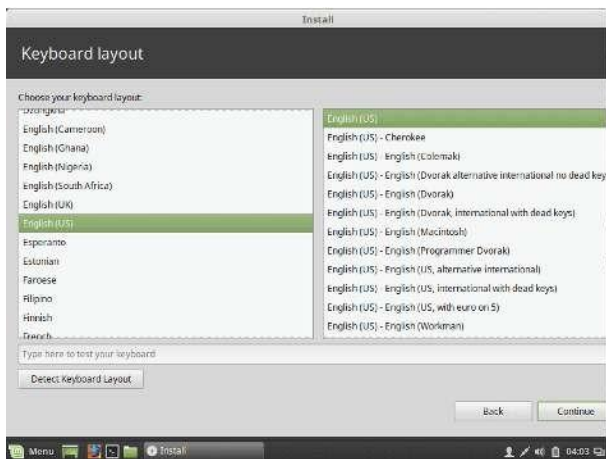
Then Create the swap area:



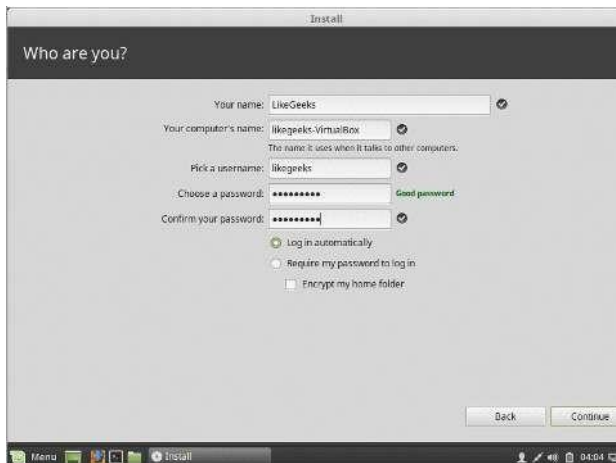
Then click install now and agree about writing changes to disk:



Now you choose the time zone and click continue the choose the language:



Now you write your username and password and click continue:



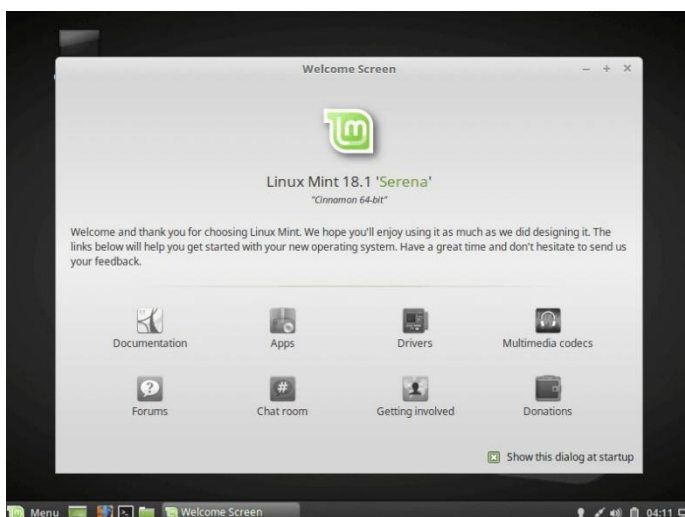
Finally, the installation started:



After finishing the installation, it will prompt you to reboot the machine and remove the installation media, whether it is a DVD or USB.



And yes, this is how to install Linux on Physical machine.





The second way to install Linux is to install it on VirtualBox. First, download VirtualBox from here:

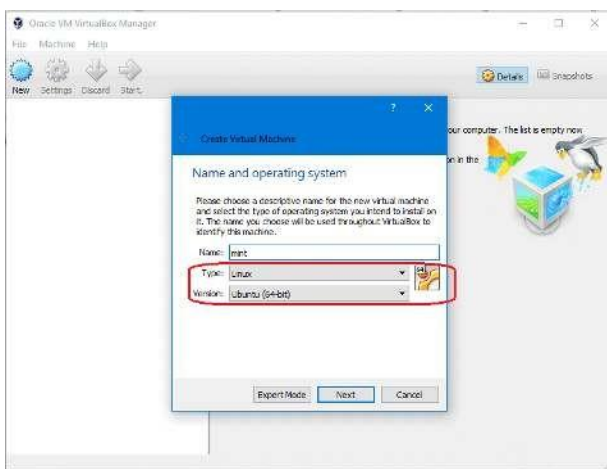
<https://www.virtualbox.org/wiki/Downloads>

There are two ways to use Linux on VirtualBox:

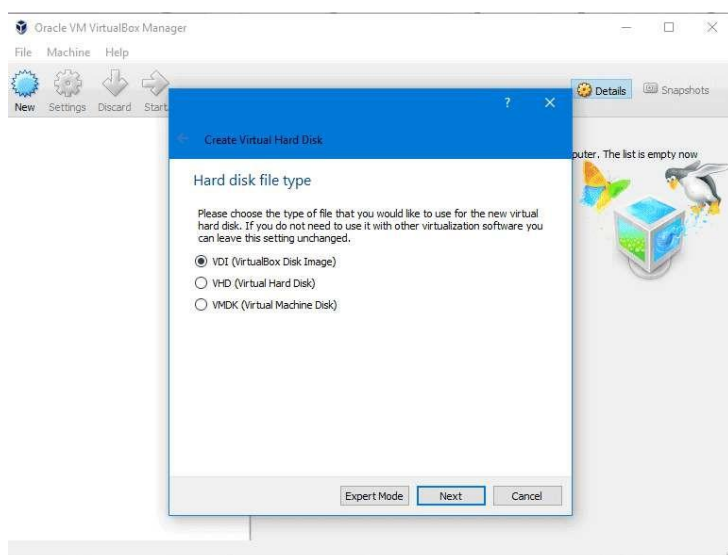
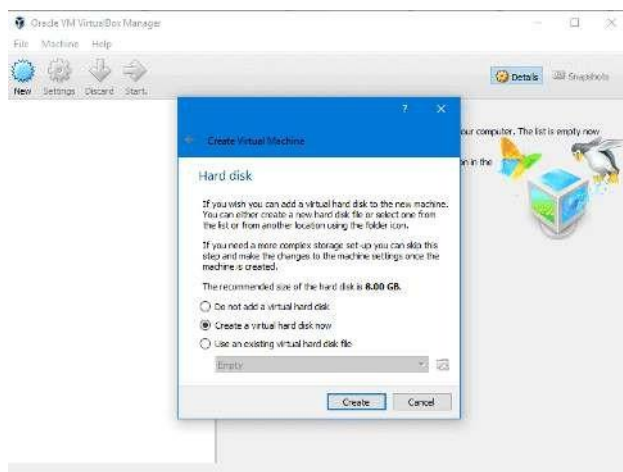
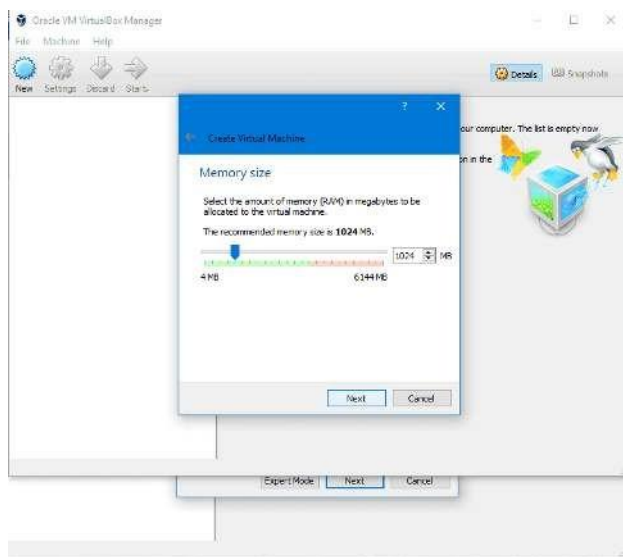
- Install it from the ISO file we downloaded above
- Download a ready to use VirtualBox image that contains the system and run it without effort from here: <http://www.osboxes.org/virtualbox-images/>

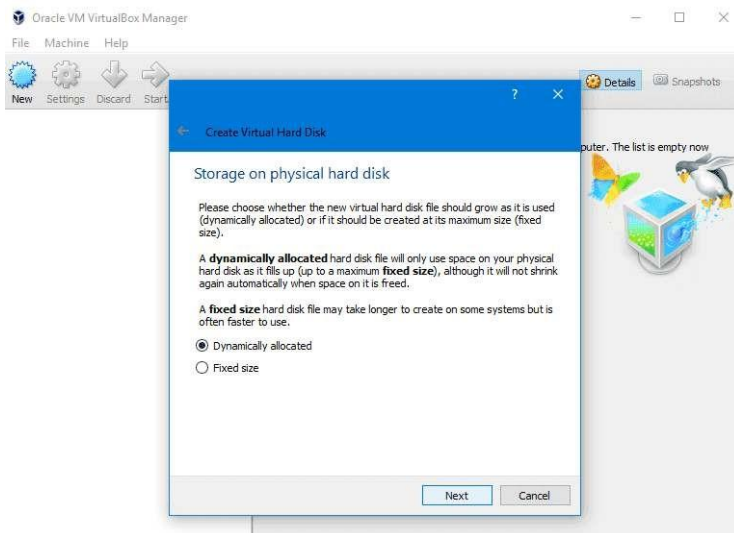
The first way is easy, it's like the normal installation process.

Open VirtualBox and click new and choose Linux and Ubuntu 64:

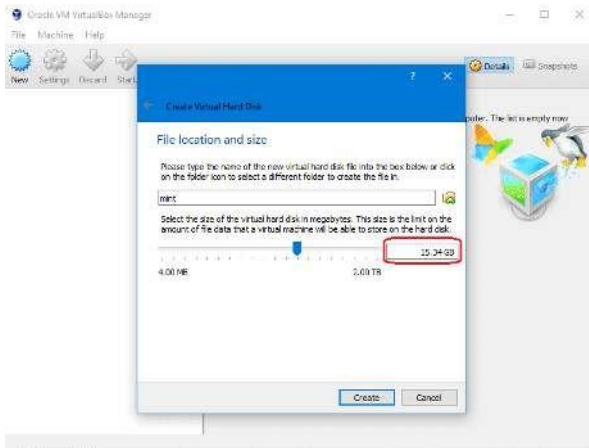


Then select the RAM required not less than 1 GB and select the disk file type or leave it as VDI and dynamically allocated, and the size not less than 12 GB and hit OK.



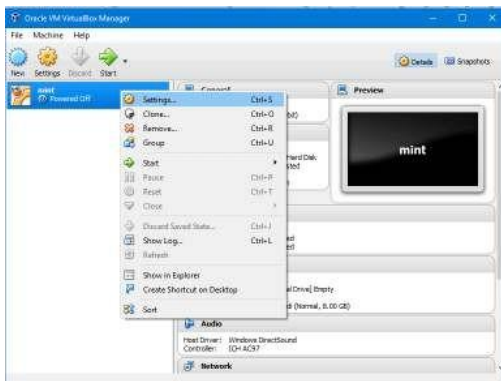


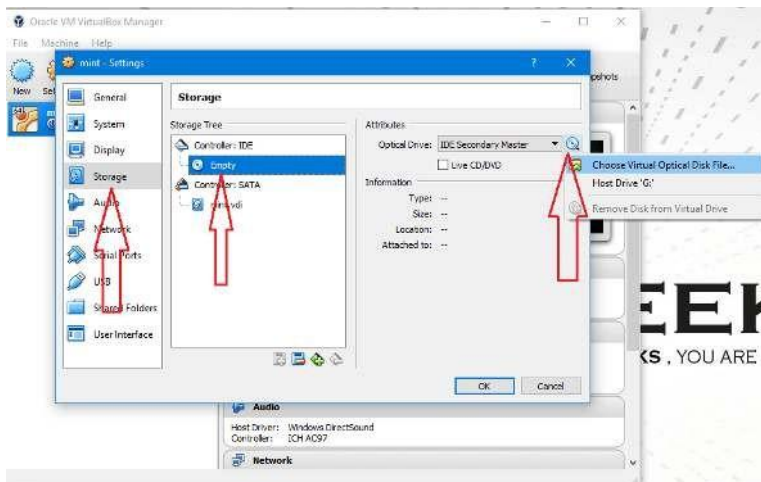
Then file location for the disk to be used.



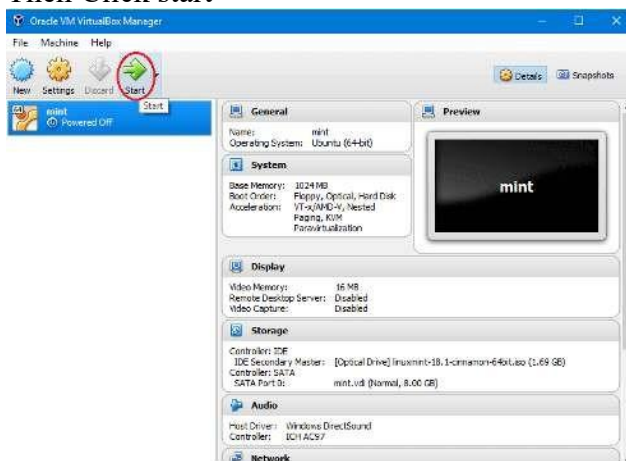
So now the VirtualBox is created, we just need to make it boot from the DVD that we've downloaded.

Choose from settings > Storage and choose the ISO image and click OK.





Then choose the iso image
Then Click start



After loading the desktop, click install Linux mint, and the rest of the steps are the same as the above mentioned without any change, And this is how to install Linux on a virtual machine.

Result:

Thus the operating system was installed successfully.

UNIX:

It is a multi-user operating system. Developed at AT & T Bell Industries, USA in 1969.

Ken Thomson along with Dennis Ritchie developed it from MULTICS (Multiplexed Information and Computing Service) OS.

By 1980, UNIX had been completely rewritten using C language.

LINUX:

It is similar to UNIX, which is created by Linus Torvalds. All UNIX commands work in Linux. Linux is an open source software. The main feature of Linux is coexisting with other OS such as Windows and UNIX.

STRUCTURE OF A LINUX SYSTEM:

It consists of three parts.

- a) UNIX kernel
- b) Shells
- c) Tools and Applications

UNIX KERNEL:

Kernel is the core of the UNIX OS. It controls all tasks, schedules all processes and carries out all the functions of OS.

Decides when one program stops and another starts.

SHELL:

Shell is the command interpreter in the UNIX OS. It accepts commands from the user and analyses and interprets them.

AIM:

To study of Basic UNIX Commands and various UNIX editors such as vi, ed, ex and EMACS.

CONTENT:

Note: Syn->Syntax

a) **date**—used to check the date and time Syn:\$date

Form	Purpose	Example	Result
+%m	To display only month	\$date+%m	06
+%h	To display month name	\$date+%h	June
+%d	To display day of month	\$date+%d	01
+%y	To display last two digits of years	\$date+%y	09
+%H	To display hours	\$date+%H	10
+	To display minutes	\$date+%M	45
+%S	To display seconds	\$date+%S	55

b) **cal**—used to display the calendar Syn:\$cal2
2009

c) **echo**—used to print the message on the screen.
Syn:\$echo“text”

d) **ls**—used to list the files. Your files are kept in a directory.
Syn:\$ls-s

All files (include files with prefix)

ls-l Lodetai (provide file statistics)

ls-t Order by creation time

ls- u Sort by access time (or show when last accessed together with -l)

ls-s Order by size

ls-r Reverse order

ls-f Mark directories with /,executable with * , symbolic links with @, local sockets with =,
named pipes(FIFOs)with

ls-s Show file size

ls- h“ Human Readable”, show file size in Kilo Bytes & Mega Bytes (h can be used together with -l or)

ls[a-m]*List all the files whose name begin with alphabets From ‘a’ to ‘m’

ls[a]*List all the files whose name begins with ‘a’ or ‘A’

Eg:\$ls>my list Output of ‘ls’ command is stored to disk file named ‘my list’

e) **lp**—used to take printouts

Syn:\$lp filename

f) **man**—used to provide manual help on every UNIX commands.

Syn:\$man unix command

\$man cat

g) **who & whoami**—it displays data about all users who have logged into the system currently.The next command displays about current user only.

Syn:\$who\$whoami

h) **uptime**—tells you how long the computer has been running since its last reboot or power-off.

Syn: \$uptime

i) **uname**—it displays the system information such as hardware platform, system name and processor, OS type.

Syn: \$uname -a j) **hostname**—

displays and sets system hostname Syn: \$hostname

k) **bc**—stands for ‘best calculator’

\$bc	\$ bc	\$ bc	\$ bc
10/2*3	scale=1	ibase=2	sqrt(196)
15	2.25+1	obase=16	14 quit 3.35
	11010011		
quit		89275	
1010		Ã	
Quit			
\$bc	\$ bc-l		
for(i=1;i<3;i=i+1)I	scale=2		
1	s(3.14)		
2	0		
3 quit			

FILE MANIPULATION COMMANDS

a) **cat**—this create, view and concatenate files.

Creation:

Syn: \$cat >filename

Viewing:

Syn: \$cat filename

Add text to an existing file:

Syn: \$cat >>filename

Concatenate:

Syn: \$cat file1 file2 > file3

\$cat file1 file2 >> file3 (no over writing of file3)

b) **grep**—used to search a particular word or pattern related to that word from the file.

Syn: \$grep search word filename

Eg: \$grep anu student

c) **rm**—deletes a file from the file system

Syn: \$rm filename

d) **touch**—used to create a blank file.

Syn: \$touch file names

e) **cp**—copies the files or directories

Syn: \$cp source file destination file

Eg: \$cp student stud

f) **mv**—to rename the file or directory
syn:\$mv old file new file
Eg:\$mv-i student student list(-i prompt when overwrite)

g) **cut**—it cuts or pickup a given number of character or fields of the file.
Syn:\$cut<option><filename>
Eg: \$cut -c filename
\$cut-c1-10emp
\$cut-f 3,6emp
\$ cut -f 3-6 emp
-c cutting columns
-f cutting fields

h) **head**—displays 10 lines from the head(top) of a given file
Syn:\$head filename
Eg:\$head student
To display the top two lines:
Syn:\$head-2student

i) **tail**—displays last 10 lines of the file
Syn:\$tail filename
Eg:\$tail student
To display the bottom two lines;
Syn:\$ tail -2 student

j) **chmod**—used to change the permissions of a file or directory.
Syn:\$ch mod category operation permission file
Where, Category—is the user type
Operation—is used to assign or remove permission
Permission—is the type of permission
File—are used to assign or remove permission all

Category	Operation	Permission
u— users	+assign	r— read
g—group	-remove	w— write
o— others	=assign absolutely	x—execute

Examples:
\$chmodu-wx student
Removes write and execute permission for users
\$ch modu+rw,g+rwestudent
Assigns read and write permission for users and groups
\$chmodg=rwx student
Assigns absolute permission for groups of all read, write and execute permissions

k) **wc**—it counts the number of lines, words, character in a specified file(s)
with the options as-l,-w,-c
Syn: \$wc-lfilename
\$wc-wfilename
\$wc-cfilename

AIM:

To study of various UNIX editors such as vi, ed, ex and EMACS.

CONCEPT:

Editor is a program that allows user to see a portions a file on the screen and modify characters and lines by simply typing at the current position. UNIX supports variety of Editors. They are:

ed ex vi

EMACS

Vi- vi is stands for “visual”.vi is the most important and powerful editor.vi is a full screen editor that allows user to view and edit entire document at the same time.vi editor was written in the University of California, at Berkley by Bill Joy, who is one of the co-founder of Sun Microsystems.

Features of vi:

It is easy to learn and has more powerful features.

It works great speed and is sensitive.vi has powerful undo functions and has 3 modes:

1. Command mode
2. Insert mode
3. Escape or ex mode

In command mode, no text is displayed on the screen.

In Insert mode, it permits user to edit insert or replace text.

In escape mode, it displays commands at command line.

Moving the cursor with the help of h, l, k, j, I, etc

EMACS EditorMotion Commands:

M-> Move to end of file

M-< Move to beginning of file

C-v Move forward a screen M -v Move

backward a screen C -n Move to next line

C-p Move to previous line

C-a Move to the beginning of the line

C-e Move to the end of the line

C-f Move forward a character

C-b Move backward a character

M-f Move forward a word

M-b Move backward a word

Deletion Commands:

DEL delete the previous character C -d delete the current character M -DEL delete the previous word

M-d delete the next word

C-x DEL deletes the previous sentence

M-k delete the rest of the current sentence

C-k deletes the rest of the current line

C-xu undo the last change

Search and Replace in EMACS:

y Change the occurrence of the pattern

n Don't change the occurrence, but look for the other q Don't change. Leave query replace completely

! Change this occurrence and all others in the file

RESULT:

Thus the various concepts such as basic commands and UNIX editors such as vi, ed, ex and EMACS were studied.

AIM:

To write simple shell programs by using conditional, branching and looping statements.

1. Write a Shell program to check the given number is even or odd**ALGORITHM:**

SEPT 1: Start the program.

STEP 2: Read the value of n.

STEP 3: Calculate 'r=expr \$n%2'.

STEP 4: If the value of r equals 0 then print the number is even

STEP 5: If the value of r not equal to 0 then print the number is odd.

PROGRAM:

```
echo "Enter the Number"
read n
r=`expr $n % 2`
if [ $r -eq 0 ]
then
echo "$n is Even number"
else
echo "$n is Odd number"
fi
```

OUTPUT

```
[nehru@localhost shellexam]$ sh oddeven
Enter the Number
35
35 is Odd number
[nehru@localhost shellexam]$ sh oddeven
Enter the Number
42
42 is Even number
```

2. Write a Shell program to check the given year is leap year or not**ALGORITHM:**

SEPT 1: Start the program.

STEP 2: Read the value of year.

STEP 3: Calculate 'b=expr \$y%4'.

STEP 4: If the value of b equals 0 then print the year is a leap year

STEP 5: If the value of r not equal to 0 then print the year is not a leap year.

PROGRAM:

```
echo "Enter the year"
read y
b=`expr $y % 4`
if [ $b -eq 0 ]
then
echo "$y is a leap year"
```

```
else
echo "$y is not a leap year"
fi
```

OUTPUT

```
[nehru@localhost shellexam]$ sh leapyear
Enter the year
2000
2000 is a leap year
[nehru@localhost shellexam]$ sh leapyear
Enter the year
2010
2010 is not a leap year
```

3. Write a Shell program to find the factorial of a number

ALGORITHM:

SEPT 1: Start the program.
STEP 2: Read the value of n.
STEP 3: Calculate 'i=expr \$n-1'.
STEP 4: If the value of i is greater than 1 then calculate 'n=expr \$n * \$i' and 'i=expr \$i - 1'.
STEP 5: Print the factorial of the given number.

PROGRAM:

```
echo "Enter a Number"
read n
i=`expr $n - 1`
p=1
while [ $i -ge 1 ]
do
n=`expr $n \* $i`
i=`expr $i - 1`
done
echo "The Factorial of the given Number is $n"
```

OUTPUT

```
[nehru@localhost shellexam]$ sh factorial
Enter a Number
6
The Factorial of the given Number is 720
```

4. Write a Shell program to swap the two integers

ALGORITHM:

SEPT 1: Start the program.
STEP 2: Read the value of a,b.
STEP 3: Calculate the swapping of two values by using a temporary variable temp.
STEP 4: Print the value of a and b.

PROGRAM:

```
echo "Enter Two Numbers"
read a b
temp=$a
a=$b
b=$temp
echo "after swapping"
echo $a $b
```

OUTPUT

```
[nehru@localhost shellexam]$ sh swaptemp
Enter Two Numbers
22 33
after swapping
33 22
```

RESULT:

Thus the shell programs written using testing and loops were executed successfully.

Ex.No.3: IMPLEMENTATION OF FORK, EXEC, GETPID, EXIT, WAIT, AND CLOSE SYSTEM CALLS.

AIM:

To write a program for implementing process management using the following system calls of UNIX operating system: fork, exec, getpid, exit, wait, close.

ALGORITHM:

Step1: Start the program.

Step2: Read the input from the command line.

Use fork() system call to create process, getppid() system call used to get the parent process ID and getpid() system call used to get the current process ID
execvp() system call used to execute that command given on that command line
argument execlp() system call used to execute specified command.

Step3: Open the directory at specified in command line input.

Step4: Display the directory contents.

Step5: Stop the program.

PROGRAM:

```
#include<stdio.h> main(int arc,char*ar[])
{
int pid; char s[100]; pid=fork();
if(pid<0)

    printf("error"); else
if(pid>0)

{
wait(NULL);

printf("\n Parent Process:\n"); printf("\n\tParent Process id:%d\t\n",getpid()); execlp("cat","cat",ar[1],(char*)0);

error("can't execute cat %s",ar[1]);
}
else
{

printf("\nchild process:");
printf("\n\tchild parent id:\t%d",getppid());
printf(s,"\n\tchild process id:\t%d",getpid());
write(1,s,strlen(s));
```

```
printf(" ");  
printf(" ");  
printf(" ");  
execvp(ar[2],&ar[2]);  
error("can't execute  
%s",ar[2]);  
  
}  
}
```

OUTPUT:

[root@localhost ~]# ./a.out tst date Child process:

Child process id :

3137 Sat Apr 10 02:45:32 IST 2010

Parent Process:

Parent Process id:3136 sd

dsaASD[root@localhost ~]# cat tst

sd dsaASD

RESULT:

Thus the program for process management was written and successfully executed

Ex. No: 4a**CPU SCHEDULING - PRIORITY****AIM:**

To write a C program for implementation of Priority scheduling algorithms.

ALGORITHM:

Step 1: Inside the structure declare the variables.

Step 2: Declare the variable i,j as integer, totwtime and totttime is equal to zero.

Step 3: Get the value of 'n' assign p and allocate the memory.

Step 4: Inside the for loop get the value of burst time and priority.

Step 5: Assign wtime as zero .

Step 6: Check p[i].pri is greater than p[j].pri .

Step 7: Calculate the total of burst time and waiting time and assign as turnaround time.

Step 8: Stop the program.

PROGRAM:**//PRIORITY SCHEDULING:**

```
#include<stdio.h>
#include<stdio.h>
#include<stdlib.h>
typedef struct
{
int pno;
int pri;
int pri;
int btime;
int wtime;
} sp;
int main()
{
int i,j,n;
int tbm=0,totwtime=0,totttime=0;
sp *p,t;
printf("\n PRIORITY SCHEDULING.\n");
printf("\n enter the no of process....\n");
scanf("%d",&n);
p=(sp*)malloc(sizeof(sp));
printf("enter the burst time and priority:\n");
for(i=0;i<n;i++)
{
printf("process%d:",i+1);
scanf("%d%d",&p[i].btime,&p[i].pri);
p[i].pno=i+1;
p[i].wttime=0;
}
for(i=0;i<n-1;i++)
for(j=i+1;j<n;j++)
```

```

{
if(p[i].pri>p[j].pri)
{
t=p[i];
p[i]=p[j];
p[j]=t;
}
}
printf("\n process\tbursttime\twaiting time\tturnaround time\n");
for(i=0;i<n;i++)
{
totwtime+=p[i].wtime=tbm;
tbm+=p[i].btime;
printf("\n%d\t\t%d",p[i].pno,p[i].btime);
printf("\t\t%d\t\t%d",p[i].wtime,p[i].wtime+p[i].btime);
}
totttime=tbm+totwtime;
printf("\n total waiting time:%d",totwtime);
printf("\n average waiting time:%f",(float)totwtime/n);
printf("\n total turnaround time:%d",totttime);
printf("\n avg turnaround time:%f",(float)totttime/n);
}

```

OUTPUT:

PRIORITY SCHEDULING.

enter the no of process....

3

enter the burst time and priority:

process1:10 2

process2:5 3

process3:2 1

process	bursttime	waiting time	turnaround time
3	2	0	2
1	10	2	12
2	5	12	17

Total waiting time:14

Average waiting time:4.666667

Total turnaround time:31

Avg turnaround time:10.333333

RESULT:

Thus the implementation of priority scheduling algorithms was executed successfully.

Ex. No: 4b**CPU SCHEDULING - ROUND ROBIN SCHEDULING****AIM:**

To write a C program for the implementation of Round Robin scheduling algorithms.

ALGORITHM:

Step 1: Inside the structure declare the variables.

Step 2: Declare the variable i,j as integer, totwtime and totttime is equal to zero.

Step 3: Get the value of 'n' assign p and allocate the memory.

Step 4: Inside the for loop get the value of burst time and priority and read the time quantum.

Step 5: Assign wtime as zero.

Step 6: Check p[i].pri is greater than p[j].pri .

Step 7: Calculate the total of burst time and waiting time and assign as turnaround time.

Step 8: Stop the program.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
struct rr
{
int pno,btime,sbtime,wtime,lst;
}p[10];
int main()
{
int pp=-1,ts,flag,count,ptm=0,i,n,twt=0,totttime=0;
printf("\n round robin scheduling..... ");
printf("enter no of processes:");
scanf("%d",&n);
printf("enter the time slice:");
scanf("%d",&ts);
printf("enter the burst time");
for(i=0;i<n;i++)
{
printf("\n process%d\t",i+1);
scanf("%d",&p[i].btime);
p[i].wtime=p[i].lst=0;
p[i].pno=i+1;
p[i].sbtime=p[i].btime;
}
printf("scheduling.... \n");
do
{
flag=0;
for(i=0;i<n;i++)
{
count=p[i].btime;
if(count>0)
```

```

{
flag=-1;
count=(count>=ts)?ts:count;
printf("\n process %d",p[i].pno);
printf("from%d",ptm);
ptm+=count; printf("to%d",ptm);
p[i].btime-=count;
if(pp!=i)
{
pp=i;
p[i].wtime+=ptm-p[i].lst-count;
p[i].lst=ptm;
}
}
}

```

OUTPUT:

ROUND ROBIN SCHEDULING.....

enter no of processes:5

enter the time slice:5

enter the burst time

process1 6

process2 7

process3 5

process4 7

process5 3

scheduling....

process 1 from 0 to5

process 2 from 5 to10

process 3 from 10 to15

process 4 from 15 to 20

process 5 from 20 to 23

process 1 from 23 to 24

process 2 from 24 to 26

process 4 from 26 to 28

process	wtime	ttime
1	18	24
2	19	26
3	10	15
4	21	28
5	20	23

total waiting time:88

average waiting time:17.600000

total turn around time:116

avg turn around time:23.200000

RESULT:

Thus the implementation of round robin scheduling algorithms was executed successfully.

Ex. No: 4c**CPU SCHEDULING - FCFS****AIM:**

To write a C program for the implementation of FCFS scheduling algorithms.

ALGORITHM:

Step 1: Inside the structure declare the variables.

Step 2: Declare the variable i,j as integer,totwtime and totttime is equal to zero.

Step 3: Get the value of 'n' assign pid as I and get the value of p[i].btime.

Step 4: Assign p[0] wtime as zero and tot time as btime and inside the loop calculate wait time and turnaround time.

Step 5: Calculate total wait time and total turnaround time by dividing by total number of process.

Step 6: Print total wait time and total turnaround time.

Step 7: Stop the program.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
struct fcfs
{
int pid;
int btime;
int wtime;
int ttime;
}
p[10];
int main()
{
int i,n;
int towtwtime=0,totttime=0;
printf("\n fcfs scheduling...\n");
printf("enter the no of process");
scanf("%d",&n);
for(i=0;i<n;i++)
{
p[i].pid=1;
printf("\n burst time of the process");
scanf("%d",&p[i].btime);
}
p[0].wttime=0;
p[0].ttime=p[0].btime;
totttime+=p[0].ttime;
for(i=0;i<n;i++)
{
p[i].wttime=p[i-1].wttime+p[i-1].btim
p[i].ttime=p[i].wttime+p[i].btime;
totttime+=p[i].ttime;
```

```

totwtime+=p[i].wtime;
}
for(i=0;i<n;i++)
{
    printf("\n waiting time for process");
    printf("\n turn around time for process");
    printf("\n");
}
printf("\n total waiting time :%d", totwtime );
printf("\n average waiting time :%f",(float)totwtime/n);
printf("\n total turn around time :%d",totttime);
printf("\n average turn around time: :%f",(float)totttime/n);
}

```

OUTPUT:

FCFS SCHEDULING...

```

enter the no of process5
burst time of the process 1  4
burst time of the process 2  2
burst time of the process 3  1
burst time of the process 4  0
burst time of the process 5  5
waiting time for process 1=0
turn around time for process 1=4
waiting time for process 2=4
turn around time for process 2=6
waiting time for process 3=6
turn around time for process 3=7
waiting time for process 4=7
turn around time for process 4=7
waiting time for process 5=7
turn around time for process 5=12
total waiting time:24
average waiting time :4.800000
total turn around time :36
average turn around time :7.200000

```

RESULT:

Thus the implementation of FCFS scheduling algorithms was executed successfully.

Ex. No: 4d**CPU SCHEDULING - SJF SCHEDULING****AIM:**

To write a C program for implementation of SJF scheduling algorithms.

ALGORITHM:

Step 1: Inside the structure declare the variables.

Step 2: Declare the variable i,j as integer,totwtime and totttime is equal to zero.

Step 3: Get the value of 'n' assign pid as I and get the value of p[i].btime.

Step 4: Assign p[0] wtime as zero and tot time as btime and inside the loop calculate wait time and turnaround time.

Step 5: Calculate total wait time and total turnaround time by dividing by total number of process.

Step 6: Print total wait time and total turnaround time.

Step 7: Stop the program.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
typedef struct
{
    int pid;
    int btime;
    int wtime;
}
sp;
int main()
{
    int i,j,n,tbm=0,towtwtime=0,totttime
    sp*p,t;
    printf("\n sjf schaduling ..\n");
    printf("enter the no of processor");
    scanf("%d",&n);
    p=(sp*)malloc(sizeof(sp));
    printf("\n enter the burst time");
    for(i=0;i<n;i++)
    {
        printf("\n process %d\t",i+1);
        scanf("%d",&p[i].btime);
        p[i].pid=i+1;
        p[i].wttime=0;
    }
    for(i=0;i<n;i++)
    for(j=j+1,j<n;j++)
    {
        if(p[i].btime>p[j].btime)
        {
            t=p[i];
```

```

p[i]=p[j];
p[j]=t;
}}
printf("\n process scheduling\n");
printf("\n process \tburst time \t w
for(i=0;i<n;i++)
{
towntime+=p[i].wtime=tbm;
tbm+=p[i].btime;
printf("\n%d\t\t%d",p[i].pid,p[i].bt
printf("\t\t%d\t\t%d",p[i].wtime,p[i]
}
tottime=tbm+towntime;
printf("\n total waiting time :%d", towttime );
printf("\n average waiting time :%f",(float)totwttime/n);
printf("\n total turn around time :%d",totttime);
printf("\n average turn around time: :%f",(float)totttime/n);
}

```

OUTPUT:

SJF SCHEDULING...

```

enter the no of process:3
burst time of the process 1  10
burst time of the process 2  4
burst time of the process 3  2
waiting time for process 1=0
turn around time for process 1=10
waiting time for process 2=10
turn around time for process 2=14
waiting time for process 3=14
turn around time for process 3=16
total waiting time:24
average waiting time :8.000000
total turn around time :40
average turn around time :13.333333

```

RESULT:

Thus the implementation of SJF scheduling algorithm was executed successfully.

Ex.No:5 IMPLEMENTATION OF IPC USING SHARED MEMORY

AIM:

To write a c program to implement IPC using shared memory.

ALGORITHM:

- Step 1: Start the process
- Step 2: Declare the segment size
- Step 3: Create the shared memory
- Step 4: Read the data from the shared memory
- Step 5: Write the data to the shared memory
- Step 6: Edit the data
- Step 7: Stop the process

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<sys/ipc.h>
#include<sys/shm.h>
#include<sys/types.h>
#define SEGSIZE 100
int main(int argc, char *argv[ ])
{
    int shmid,cntr;
    key_t key;
    char *segptr;
    char buff[]="poooda..... ";
    key=ftok(".", 's');
    if((shmid=shmget(key, SEGSIZE, IPC_CREAT | IPC_EXCL | 0666))== -1)
    {
        if((shmid=shmget(key,SEGSIZE,0))== -1)
        {
            perror("shmget");
            exit(1);
        }
    }
    else
    {
        printf("Creating a new shared memory seg \n");
        printf("SHMID:%d",shmid);
    }
    system("ipcs -m");
    if((segptr=(char*)shmat(shmid,0,0))== (char*)-1)
    {
        perror("shmat");
        exit(1);
    }
```

```

}
printf("Writing data to shared memory...\n");
strcpy(segptr,buff);
printf("DONE\n");
printf("Reading data from shared memory...\n");
printf("DATA:-%s\n",segptr);
printf("DONE\n");
printf("Removing shared memory Segment...\n");
if(shmctl(shmid,IPC_RMID,0)== -1)
printf("Can't Remove Shared memory Segment...\n");
else
printf("Removed Successfully");
}

```

OUTPUT:

Creating new shared memory segments

Key	Scmid	Owner	Bytes
0x006ff429	0	postgres	3240725
0x00000000	32729	root	393216
0x00000000	32629	root	393216
0x00000000	98730	root	393216

Semaphored arrays

Key	Scmid	nsems
0x006ff429	0	17
0x00000000	32729	17
0x00000000	32629	17
0x00000000	98730	17

RESULT:

Thus the algorithm to implementation IPC using shared memory was executed successfully.

Ex.No.:6**Implement mutual exclusion by Semaphores****AIM:**

To write a c program for the implementation of mutual exclusion by Semaphores.

ALGORITHM:

Step 1: The Semaphore mutex, full & empty are initialized.

Step 2: In the case of producer process

- i) Produce an item in to temporary variable.
- ii) If there is empty space in the buffer checks the mutex value for enter into the critical section.
- iii) If the mutex value is 0, allow the producer to add value in the temporary variable to the buffer.

Step 3: In the case of consumer process

- i) It should wait if the buffer is empty
- ii) If there is any item in the buffer check formutex value, if the mutex==0,remove item from buffer
- iii) Signal the mutex value and reducetheemptyvalueby1.

Step 4: Consume the item.

Step 5: Print the result

PROGRAM:

```
#include<stdio.h>
#include<pthread.h>
typedef struct
{
    pthread_mutex_t mutex;
    pthread_cond_t condition;
    int value;
} semaphore;
void semaphore_init(semaphore* sem,int
initial_value){ pthread_mutex_init(&(sem-
>mutex),NULL); pthread_cond_init(&(sem-
>condition),NULL);
sem->value=initial_value;
}

void semaphore_wait(semaphore*
sem){ pthread_mutex_lock(&(sem-
>mutex)); while(sem->value<=0){
pthread_cond_wait(&(sem->condition),&(sem->mutex));
}
Sem->value--;
Pthread_mutex_unlock(&(sem->mutex));
}
```

```

void semaphore_signal(semaphore* sem)
{
pthread_mutex_lock(&(sem->mutex));
sem->value++;
pthread_cond_signal(&(sem->condition));
pthread_mutex_unlock(&(sem->mutex));
}
void* thread_function(void*
arg){ semaphore*
sem=(semaphore*)arg;
printf("Thread waiting\n");
semaphore_wait(sem);
printf("thread acquired the semaphore\n");
printf("thread releasing the semaphore\n");
semaphore_signal(sem);
return NULL;
}
int
main(){ semaph
ore sem;
semaphore_init(&sem,1);
pthread_t thread;
pthread_create(&thread,NULL,thread_function,(void*)&sem);
printf("main thread performing some work\n");
printf("main thread waiting\n");
semaphore_wait(&sem);
printf("main thread acquired the semaphore\n");
printf("main thread releasing the semaphore\n");
semaphore_signal(&sem);
pthread_join(thread,NULL);
return 0;
}

```

OUTPUT:

```

Main thread performing some work
Main thread waiting
Main thread acquired the semaphore
Main thread releasing the semaphore
Thread waiting
Thread acquired the semaphore
Thread releasing the semaphore

```

RESULT:

Thus the c program to implement mutual exclusion by using semaphores was executed successfully.

Ex.No :7 IMPLEMENTATION OF BANKERS ALGORITHM FOR DEADLOCK AVOIDANCE

AIM:

To write a C program to implement banker's algorithm for deadlock avoidance.

ALGORITHM:

Step-1: Start the program.

Step-2: Declare the memory for the process.

Step-3: Read the number of process, resources, allocation matrix and available matrix.

Step-4: Compare each and every process using the banker's algorithm.

Step-5: If the process is in safe state then it is not a deadlock process otherwise it is a deadlock process

Step-6: Produce the result of state of process

Step-7: Stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
int max[100][100];
int alloc[100][100];
int need[100][100];
int avail[100];
int n,r;
void input();
void show();
void cal();
int main()
{
    int i,j;
    printf("***** Baner's Algo *****\n");
    input();
    show();
    cal();
    getch();
    return 0;
}
void input()
{
    int i,j;
    printf("Enter the no of Processes\t");
    scanf("%d",&n);
    printf("Enter the no of resources instances\t");
    scanf("%d",&r);
    printf("Enter the Max Matrix\n");
    for(i=0;i<n;i++)
    {
```

```

for(j=0;j<r;j++)
{
scanf("%d",&max[i][j]);
}}
printf("Enter the Allocation Matrix\n");
for(i=0;i<n;i++)
{
for(j=0;j<r;j++)
{
scanf("%d",&alloc[i][j]);
}}
printf("Enter the available Resources\n");
for(j=0;j<r;j++)
{
scanf("%d",&avail[j]);
}}
void show()
{
int i,j;
printf("Process\t Allocation\t Max\t Available\t");
for(i=0;i<n;i++)
{
printf("\nP%d\t ",i+1);
for(j=0;j<r;j++)
{
printf("%d ",alloc[i][j]);
}
printf("\t");
for(j=0;j<r;j++)
{
printf("%d ",max[i][j]);
}
printf("\t");
if(i==0)
{
for(j=0;j<r;j++)
printf("%d ",avail[j]);
}}
}
void cal()
{
int finish[100],temp,need[100][100],flag=1,k,c1=0;
int safe[100];
int i,j;
for(i=0;i<n;i++)
{
finish[i]=0;
}
//find need matrix
for(i=0;i<n;i++)
{

```

```

for(j=0;j<r;j++)
{
need[i][j]=max[i][j]-alloc[i][j];
}}
printf("\n");

while(flag)
{
flag=0;
for(i=0;i<n;i++)
{
int c=0;
for(j=0;j<r;j++)
{
if((finish[i]==0)&&(need[i][j]<=avail[j]))
{ c+
+;
if(c==r)
{
for(k=0;k<r;k++)
{
avail[k]+=alloc[i][j];
finish[i]=1;
flag=1;
}
printf("P%d->",i);
if(finish[i]==1)
{
i=n;
}}}}}}
for(i=0;i<n;i++)
{
if(finish[i]==1)
{ c1+
+;
}
Else
{printf("P%d->",i);
}}
if(c1==n)
{printf("\n The system is in safe state");
}
Else
{
printf("\n Process are in dead lock");
printf("\n System is in unsafe state");
}}

```

OUTPUT:

Enter the no of Processes 5
Enter the no of resources instances 3
Enter the Max Matrix
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter the Allocation Matrix
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter the available Resources
3 3 2

Process	Allocation	Max	Available
P1	0 1 0	7 5 3	3 3 2
P2	2 0 0	3 2 2	
P3	3 0 2	9 0 2	
P4	2 1 1	2 2 2	
P5	0 0 2	4 3 3	

P1->P3->P4->P2->P0->
The system is in safe state

RESULT:

Thus the banker's algorithm for avoiding deadlock has been implemented successfully.

Ex .No:8 IMPLEMENTATION OF ALGORITHM FOR DEADLOCK DETECTION

AIM:

To write a C program to implement algorithm for deadlock detection.

ALGORITHM:

Step-1: Start the program.

Step-2: Declare the memory for the process.

Step-3: Read the number of process, resources, allocation matrix and available matrix.

Step-4: Compare each and every process using the banker's algorithm.

Step-5: If the process is in safe state then it is not a deadlock process otherwise it is a deadlock process

Step-6: Produce the result of state of process

Step-7: Stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
int max[100][100];
int alloc[100][100];
int need[100][100];
int avail[100];
int n,r;
void input();
void show();
void cal();
int main()
{
    int i,j;
    printf("***** Deadlock Detection Algo *****\n");
    input();
    show();
    cal();
    getch();
    return 0;
}
void input()
{int i,j;
printf("Enter the no of Processes\t");
scanf("%d",&n);
printf("Enter the no of resource instances\t");
scanf("%d",&r);
printf("Enter the Max Matrix\n");
for(i=0;i<n;i++)
{for(j=0;j<r;j++)
{
scanf("%d",&max[i][j]);
}}}
```

```

printf("Enter the Allocation Matrix\n");
for(i=0;i<n;i++)
{for(j=0;j<r;j++)
{
scanf("%d",&alloc[i][j]);
}}
printf("Enter the available Resources\n");
for(j=0;j<r;j++)
{
scanf("%d",&avail[j]);
}}
void show()
{
int i,j;
printf("Process\t Allocation\t Max\t Available\t");
for(i=0;i<n;i++)
{
printf("\nP%d\t ",i+1);
for(j=0;j<r;j++)
{
printf("%d ",alloc[i][j]);
}
printf("\t");
for(j=0;j<r;j++)
{printf("%d ",max[i][j]);
}
printf("\t");
if(i==0)
{
for(j=0;j<r;j++)
printf("%d ",avail[j]);
}}
}
void cal()
{ int finish[100],temp,need[100][100],flag=1,k,c1=0;
int dead[100];
int safe[100];
int i,j;
for(i=0;i<n;i++)
{finish[i]=0;
}
//find need matrix
for(i=0;i<n;i++)
{for(j=0;j<r;j++)
{
need[i][j]=max[i][j]-alloc[i][j];
}}
while(flag)
{flag=0;

```



```

for(i=0;i<n;i++)
{int c=0;
for(j=0;j<r;j++)
{if((finish[i]==0)&&(need[i][j]<=avail[j]))
{c++;
if(c==r)
{
for(k=0;k<r;k++)
{avail[k]+=alloc[i][j];
finish[i]=1;
flag=1;
} //printf("\nP%d",i);
if(finish[i]==1)
{i=n;
}}}}}}
j=0;
flag=0;
for(i=0;i<n;i++)
{
if(finish[i]==0)
{dead[j]=i;
j++;
flag=1;
}}
if(flag==1)
{
printf("\n\nSystem is in Deadlock and the Deadlock process are\n");
for(i=0;i<n;i++)
{printf("P%d\t",dead[i]);
}}
}
Else
{
printf("\nNo Deadlock Occur");
}}

```

OUTPUT:

*****Deadlock Detection Algo *****

Enter the no of processes 3

Enter the no of resources 3

Enter the Max Matrix

3 6 8

4 3 3

3 4 3

Enter the Allocation Matrix

3 3 3

2 0 3

1 2 4

Enter the available Resouces

1 2 0

Process	Available	Max	Available
---------	-----------	-----	-----------

P1	3 3 3	3 6 8	1 2 0
----	-------	-------	-------

P2	2 0 3	4 3 3	
----	-------	-------	--

P3	1 2 4	3 4 4	
----	-------	-------	--

System is in Deadlock and the deadlock processes are

P0 p1 p2

RESULT:

Thus the algorithm for detecting deadlock has been implemented successfully.

Ex.No.:9 IMPLEMENT THREADING APPLICATIONS

AIM:

To write a C program to implement threading applications.

ALGORITHM:

- Step 1: Start the process
- Step 2: Declare the thread and mutex locks
- Step 3: Lock the mutex lock for the process
- Step 4: Read the process and data of process
- Step 5: Release the process
- Step 6: Display the order of process execution
- Step 7: Stop the process

PROGRAM:

```
#include<stdio.h>
#include<string.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>
pthread_t tid[2];
int counter;
pthread_mutex_t lock;
void* doSomething(void *arg)
{
    pthread_mutex_lock(&lock);
    unsigned long i = 0;
    counter += 1;
    printf("\n Job %d started\n", counter);
    for(i=0; i<(0xFFFFFFFF);i++);
    printf("\n Job %d finished\n", counter);
    pthread_mutex_unlock(&lock);
    return NULL;
}
int main(void)
{
    int i = 0;
    int err;
    if (pthread_mutex_init(&lock, NULL) != 0)
    { printf("\n mutex init failed\n");
    return 1;
    }
    while(i < 2)
    {
        err = pthread_create(&(tid[i]), NULL, &doSomething, NULL);
        if (err != 0)
            printf("\ncan't create thread :[%s]", strerror(err));
        i++;
    }
}
```

```
pthread_join(tid[0], NULL);  
pthread_join(tid[1], NULL);  
pthread_mutex_destroy(&lock);  
return 0;}
```

OUTPUT:

```
$ ./threads  
Job 1 started  
Job 1 finished  
Job 2 started  
Job 2 finished
```

RESULT:

Thus the implementation of threading application was executed successfully.

Ex.No.:10 IMPLEMENTATION OF PAGING TECHNIQUE OF MEMORYMANAGEMENT

AIM:

To write a c program to implement Paging technique for memory management.

ALGORITHM:

- Step 1: Start the process
- Step 2: Declare page number, page table, frame number and process size.
- Step 3: Read the process size, total number of pages
- Step 4: Read the relative address
- Step 5: Calculate the physical address
- Step 6: Display the address
- Step 7: Stop the process

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
main()
{
int size,m,n,pagno,pahtable[3]={5,6,7},i,j,frameno;
double m1;
int ra=0,ofs;
clrscr();
printf("Enter process size (in KB of max 12KB):");/*reading memeory size*/
scanf("%d",&size);
m1=size/4;
n=ceil(m1);
printf("Total No. of pages: %d",n);
printf("\nEnter relative address (in hexadecimal notation eg.0XRA) \n");
//printf("The length of relative Address is : 16 bits \n\n The size of offset is :12 bits\n");
scanf("%d",&ra);
pagno=ra/1000; /*calculating physical address*/
ofs=ra%1000;
printf("page no=%d\n",pagno);
printf("page table");
for(i=0;i<n;i++)
printf("\n %d [%d]",i,pahtable[i]);
frameno=pahtable[pagno];
printf("\n Equivalent physical address : %d%d",frameno,ofs);
getch();
}
```

OUTPUT:

Enter process size (in KB of max 12KB): 12
Total No. of pages: 3
Enter relative address (in hexadecimal notation eg.OXRA)
2643
Page no=2
Page table
0 [5]
1 [6]
2[7]
Equivalent physical address: 7643

RESULT:

Thus the implementation of paging technique for memory management was executed successfully.

Ex.No.:11 IMPLEMENTATION OF MEMORY ALLOCATION TECHNIQUES

AIM:

To write a C program to implement Memory Management concept using the Technique best fit, worst fit and first fit algorithms.

ALGORITHM:

Step1: Get the number of process.

Step 2: Get the number of blocks and sizeof process.

Step3: Get the choices from the user and call the corresponding witch cases.

Step4: Firstfit-allocatetheprocesstotheavailablefreeblockmatchwiththesizeof the process

Step5: Worstfit-allocatetheprocesstothe largestblocksizeavailableinthelist

Step6: Bestfit-allocatetheprocesstothe optimumsizeblockavailableinthelist

Step7: Display the result with allocations

PROGRAM:

```
#include
<stdio.h>
main()
{
int p[10],np,b[10],nb,ch,c[10],d[10],alloc[10],flag[10],i,j;
printf("\nEnter the no of process:");
scanf("%d",&np);

printf("\nEnter the no of blocks:");
scanf("%d",&nb);

printf("\nEnter the size of each process:");
for(i=0;i<np;i++)
{
printf("\nProcess%d:",i);
scanf("%d",&p[i]);
}

printf("\nEnter the block sizes:");
for(j=0;j<nb;j++)
{
printf("\nBlock %d:",j);
scanf("%d",&b[j]);c[j]=b[j];d[j]=b[j];
}
}
```

```

if(np<=nb)
{
printf("\n1.Firstfit2.Bestfit3.Worstfit"); do
{
printf("\nEnter your choice:");
scanf("%d",&ch);
switch(ch)
{
case 1: printf("\nFirst Fit\n"); for(i=0;i<np;i++)
{
for(j=0;j<nb;j++)
{
if(p[i]<=b[j])
{
alloc[j]=p[i];printf("\n\nAlloc[%d]",all oc[j]);
printf("\n\nProcess %d of size %d is allocatedinblock:%dofsize:
%d",i,p[i],j,b[j]);
flag[i]=0,b[j]=0;break;
}

else
flag[i]=1;
}
}
for(i=0;i<np;i++)
{
if(flag[i]!=0)
printf("\n\nProcess %d of size %d is not allocated",i,p[i]);
}

break;
case 2: printf("\nBest Fit\n");
for(i=0;i<nb;i++)

```



```

{
for(j=i+1;j<nb;j++)
{
if(c[i]>c[j])
{
inttemp=c[i]; c[i]=c[j]; c[j]=temp;
}
}
}

printf("\nAfter sorting block sizes:"); for(i=0;i<nb;i++)
printf("\nBlock%d:%d",i,c[i]); for(i=0;i<np;i++)
{
for(j=0;j<nb;j++)
{
if(p[i]<=c[j])
{
}
else
}
}

alloc[j]=p[i];printf("\n\nAlloc[%d]",all oc[j]);
printf("\n\nProcess %d of size %d is allocated in block %d of size
%d",i,p[i],j,c[j]); flag[i]=0,c[j]=0;break;
flag[i]=1;
for(i=0;i<np;i++)
{
}
break;
if(flag[i]!=0)
printf("\n\nProcess %d of size %d is not allocated",i,p[i]);
case 3: printf("\nWorst Fit\n"); for(i=0;i<nb;i++)

```

```

{
for(j=i+1;j<nb;j++)
{
if(d[i]<d[j])
{
inttemp=d[i]; d[i]=d[j]; d[j]=temp;
}
}
}

printf("\nAfter sorting block sizes:"); for(i=0;i<nb;i++)
printf("\nBlock%d:%d",i,d[i]); for(i=0;i<np;i++)
{
for(j=0;j<nb;j++)
{
if(p[i]<=d[j])
{
alloc[j]=p[i];
printf("\n\nAlloc[%d]",alloc[j]);
printf("\n\nProcess %d of size %d is allocated in block %d of size
%d",i,p[i],j,d[j]
flag[i]=0,d[j]=0;break;
}
else
flag[i]=1;
}
if(flag[i]!=0)
printf("\n\nProcess%d of size %d is not allocated",I,p[i])
}
Printf("Invalid choice....!");
break;
default:

```

```
break;
}
for(i=0;i<np;i++)
{
}
}while(ch<=3);
}
}
```

OUTPUT:

Enter the no of
process:3 Enter
the no of
blocks:3

Enter the size of each process:

Process0:100

Process1:150

Process2:200

Enter the block sizes:

Block0:300

Block1:350

Block2:200

1.First fit 2.Best fit
3.Worst fit Enter your
choice:1

Alloc[100]

Process 0 of size 100 is allocated in block 0 of size 300

Alloc[150]

Process 1 of size 150 is allocated in block 1 of size 350

Alloc[200]

Process 2 of size 200 is allocated in block 2 of
size 200 Enter your choice:2

BestFit

After sorting block
sizes are: Block
0:200

Block1:300

Block2:350

Alloc[100]

Process 0 of size 100 is allocated in block:0 of size:200
Alloc[150]

Process 1 of size 150 is allocated in block:1 of size:300
Alloc[200]

Process 2 of size 200 is allocated in block:2 of
size:350 enter your choice:3

Worst Fit

After sorting block sizes are:

Block0:350

Block1:300

Bloc
k2:2
00
Allo
c[10
0]

Process 0 of size 100 is allocated in block 0 of size 350
Alloc[150]

Process 1 of size 150 is allocated in block 1 of size 300
Alloc[200]

Process 2 of size 200 is allocated in block 2 of
size 200 Enter your choice:6

InvalidChoice...!

RESULT:

Thus the c program to implement memory management scheme using Best fit, worst fit and first fit were executed successfully.

Ex.No:12a. IMPLEMENTATION OF FIFO PAGE REPLACEMENT ALGORITHM

AIM:

To write a C program for implementation of FIFO page replacement algorithm.

ALGORITHM:

Step 1: Start the program.

Step 2: Declare the necessary variables.

Step 3: Enter the number of frames.

Step 4: Enter the reference string end with zero.

Step 5: FIFO page replacement selects the page that has been in memory the longest time and when the page must be replaced the oldest page is chosen.

Step 6: When a page is brought into memory, it is inserted at the tail of the queue.

Step 7: Initially all the three frames are empty.

Step 8: The page fault range increases as the no of allocated frames also increases.

Step 9: Print the total number of page faults.

Step 10: Stop the program.

PROGRAM:

```
#include<stdio.h>
int main()
{
int i=0,j=0,k=0,i1=0,m,n,rs[30],flag=1,p[30];
system("clear");
printf("FIFO page replacement algorithm....\n");
printf("enter the no. of frames:");
scanf("%d",&n);
printf("enter the reference string:");
while(1)
{
scanf("%d",&rs[i]);
if(rs[i]==0)
break;
i++;
}
m=i;
for(j=0;j<n;j++)
p[j]=0;
for(i=0;i<m;i++)
{
flag=1;
for(j=0;j<n;j++)
if(p[j]==rs[i])
{
printf("data already in page....\n");
flag=0;
break;
}
```

```

}
if(flag==1)
{
p[i1]=rs[i];
i1++;
k++;
if(i1==n)
i1=0;
for(j=0;j<n;j++)
{
printf("\n page %d:%d",j+1,p[j]);
if(p[j]==rs[i])
printf("*");
}
printf("\n\n");
}
}
printf("total no page faults=%d",k);
}

```

OUTPUT:

```

FIFO page replacement algorithm....
enter the no. of frames:2
enter the reference string:3 4 6 8 1 0
page 1:3*
page 2:0
page 1:3
page 2:4*
page 1:6*
page 2:4
page 1:6
page 2:8*
page 1:1*
page 2:8
total no page faults=5

```

RESULT:

Thus the implementation of FIFO page replacement algorithms was executed successfully.

Ex.No:12b IMPLEMENTATION OF LRU PAGE REPLACEMENT ALGORITHM

AIM:

To write a c program to implement LRU page replacement algorithm.

ALGORITHM:

- Step 1: Start the process
- Step 2: Declare the size
- Step 3: Get the number of pages to be inserted
- Step 4: Get the value
- Step 5: Declare counter and stack
- Step 6: Select the least recently used page by counter value
- Step 7: Stack them according to the selection.
- Step 8: Display the values
- Step 9: Stop the process

PROGRAM:

```
#include<stdio.h>
main()
{
int q[20],p[50],c=0,c1,d,f,i,j,k=0,n,r,t,b[20],c2[20];
printf("Enter no of pages:");
scanf("%d",&n);
printf("Enter the reference string:");
for(i=0;i<n;i++) scanf("%d",&p[i]);
printf("Enter no of frames:");
scanf("%d",&f);
q[k]=p[k];
printf("\n\t%d\n",q[k]);
c++;
k++;
for(i=1;i<n;i++)
{
    c1=0;
    for(j=0;j<f;j++)
    {
        if(p[i]!=q[j])
            c1++;
    }
    if(c1==f)
    {c++;
    if(k<f)
    {q[k]=p[i];
    k++;
    for(j=0;j<k;j++)
    printf("\t%d",q[j]);
```

```

printf("\n");
}
else
{for(r=0;r<f;r++)
{c2[r]=0;
for(j=i-1;j<n;j--)
{if(q[r]!=p[j])
c2[r]++;
else
break;
}}
for(r=0;r<f;r++)
b[r]=c2[r];
for(r=0;r<f;r++)
{
for(j=r;j<f;j++)
{
if(b[r]<b[j])
{
t=b[r];
b[r]=b[j];
b[j]=t;
}}
}
for(r=0;r<f;r++)
{
if(c2[r]==b[0])
q[r]=p[i];
printf("\t%d",q[r]);
}
printf("\n");
}}
printf("\nThe no of page faults is %d",c);
}

```


OUTPUT:

Enter no of pages:10

Enter the reference string:7 5 9 4 3 7 9 6 2 1

Enter no of frames:3

7

7 5

7 5 9

4 5 9

4 3 9

4 3 7

9 3 7

9 6 7

9 6 2

1 6 2

The no of page faults is 10

RESULT:

Thus the implementation of LRU page replacement algorithms was executed successfully.

AIM:

To write C program to implement LFU page replacement algorithm.

ALGORITHM:

- Step 1: Start the process
- Step 2: Declare the size
- Step 3: Get the number of pages to be inserted
- Step 4: Get the value
- Step 5: Declare counter and stack
- Step 6: Select the least frequently used page by counter value
- Step 7: Stack them according the selection.
- Step 8: Display the values
- Step 9: Stop the process

PROGRAM:

```
#include<stdio.h>
int main()
{
    int f,p;
    int pages[50],frame[10],hit=0,count[50],time[50];
    int i,j,page,flag,least,minTime,temp;
    printf("Enter no of frames : ");
    scanf("%d",&f);
    printf("Enter no of pages : ");
    scanf("%d",&p);
    for(i=0;i<f;i++)
    {
        frame[i]=-1;
    }
    for(i=0;i<50;i++)
    {
        count[i]=0;
    }
    printf("Enter page no : \n");
    for(i=0;i<p;i++)
    {
        scanf("%d",&pages[i]);
    }
    printf("\n");
    for(i=0;i<p;i++)
    {
        count[pages[i]]++;
        time[pages[i]]=i;
        flag=1;
        least=frame[0];
```

```
for(j=0;j<f;j++)
{
if(frame[j]==-1 || frame[j]==pages[i])
{
if(frame[j]!=-1)
{
hit++;
}
flag=0;
frame[j]=pages[i];
break;
}
if(count[least]>count[frame[j]])
{
least=frame[j];
}
}
if(flag)
{
minTime=50;
for(j=0;j<f;j++)
{
if(count[frame[j]]==count[least] && time[frame[j]]<minTime)
{
temp=j;
minTime=time[frame[j]];
}
}
count[frame[temp]]=0;
frame[temp]=pages[i];
}
for(j=0;j<f;j++)
{
printf("%d ",frame[j]);
}
printf("\n");
}
printf("Page hit = %d",hit);
return 0;
}
```

OUTPUT:

Enter no of frames: 3
Enter no of pages: 10
Enter page no:
2 3 4 2 1 3 7 4 3
2 -1 -1
2 3 -1
2 3 4
2 1 4
2 1 3
2 7 3
2 7 5
2 4 5
2 4 3
Page Hit = 1

RESULT:

Thus the implementation of LFU page replacement algorithms was executed successfully.

Ex.No.:13a FILE ORGANIZATION TECHNIQUE-SINGLE LEVEL DIRECTORY

AIM:

To write C program to organize the file using single level directory.

ALGORITHM:

- Step-1: Start the program.
- Step-2: Declare the count, file name, graphical interface.
- Step-3: Read the number of files
- Step-4: Read the file name
- Step-5: Declare the root directory
- Step-6: Using the file eclipse function define the files in a single level
- Step-7: Display the files
- Step-8: Stop the program

PROGRAM:

```
#include<stdio.h> main()
{
int master,s[20]; charf[20][20][20];
chard[20][20]; int i,j;
printf("enter number of directorios:"); scanf("%d",&master);
printf("enter names of directories:"); for(i=0;i<master;i++) scanf("%s",&d[i]);
printf("enter size of directories:"); for(i=0;i<master;i++) scanf("%d",&s[i]);
printf("enter the file names:"); for(i=0;i<master;i++) for(j=0;j<s[i];j++) scanf("%s",&f[i]
[j]); printf("\n");
printf(" directory\tsize\tfilenames\n");
printf("*****\n");
for(i=0;i<master;i++)
{
printf("%s\t\t%2d\t",d[i],s[i]); for(j=0;j<s[i];j++)
printf("%s\n\t\t",f[i][j]); printf("\n");
}
printf("\t\n");
}
```

OUTPUT:

Enter the number of directories: 2
Enter the names of directories: cse it
Enter size of directories: 3 4
Enter the file names: AI
OS
ML
OCI
DS
Cloud
Blockchain

Directory	size	filename

cse	3	AI
		OS
		ML
it	4	OCI
		DS
		Cloud
		Blockchain

RESULT:

Thus the file organization technique -Single level directory structure was executed successfully.

Ex.No:13b FILE ORGANIZATION TECHNIQUE-TWO LEVEL DIRECTORY

AIM:

To write C program to organize the file using two level directory.

ALGORITHM:

- Step-1: Start the program.
- Step-2: Declare the count, file name, graphical interface.
- Step-3: Read the number of files
- Step-4: Read the file name
- Step-5: Declare the root directory
- Step-6: Using the file eclipse function define the files in a single level
- Step-7: Display the files
- Step-8: Stop the program

PROGRAM:

```
#include<stdio.h>struct st
{
char dname[10];

char sdname[10][10]; char fname[10][10][10]; int ds,sds[10];
} dir[10]; void main()
{
int i,j,k,n;

clrscr();

printf("enter number of directories:");

scanf("%d",&n);

for(i=0;i<n;i++)
{
printf("enter directory %dnames:",i+1);

scanf("%s",&dir[i].dname);

printf("enter size of directories:");

scanf("%d",&dir[i].ds);

for(j=0;j<dir[i].ds;j++)
{
printf("enter sub directory name and size:");

scanf("%s",&dir[i].sdname[j]);
```

```

scanf("%d",&dir[i].sds[j]);

for(k=0;k<dir[i].sds[j];k++)

{
printf("enter file name:");
scanf("%s",&dir[i].fname[j][k]);

}}}
printf("\ndirname\t\tsize\tsubdirname\tsize\tfiles"); printf("\n*****\n"); for(i=0;i<n;i++){

printf("%s\t\t%d",dir[i].dname,dir[i].ds);

for(j=0;j<dir[i].ds;j++)

{
printf("\t%s\t\t%d\t",dir[i].sdname[j],dir[i].sds[j]);

for(k=0;k<dir[i].sds[j];k++) printf("%s\t",dir[i].fname[j][k]);

printf("\n\t");}

printf("\n"); }

}

```

OUTPUT:

```

enter number of directories:2
enter directory 1 name:cse
enter size of directories:3
enter subdirectory name and size:os 3
enter file name:aaa
enter file name:bbb
enter file name:ccc
enter subdirectory name and size:cn 2
enter file name:xyz
enter file name:mic
enter subdirectory name and size:apt 2
enter file name:anh
enter file name:kkk
enter directory 2 name:ece
enter size of directories:2

```


enter subdirectory name and size:ct 2
enter file name:aav
enter file name:vcs
enter subdirectory name and size:dc 4
enter file name:afs
enter file name:ged

enter file name:eee
enter file name:ttt

dirname	size	subdirname	size	files			

cse	3	os	3	aaa	bbb	ccc	
		cn	2	xyz	mic		
		apt	2	anh	kkk		
ece	2	ct	2	aav	vcs		
		dc	4	afs	ged	eee	ttt

RESULT:

Thus the file organization technique -Two level directory structure was executed successfully.

AIM:

To write a C program for sequential file for processing the student information.

ALGORITHM:

Step-1: Start the program.

Step-2: Get the number of records user want to store in the system.

Step-3: Using Standard Library function open the file to write the data into the file.

Step-4: Store the entered information in the system.

Step-5: Using do..While statement and switch case to create the options such as
1-DISPLAY, 2.SEARCH, 3.EXIT.

Step-6: Close the file using fclose() function.

Step-7: Process it and display the result.

Step-8: Stop the program.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
typedef struct
{int sno;
char name[25];int m1,m2,m3;
}STD;
void display(FILE *);
int search(FILE *);
void main()
{int i,n,sno_key,opn;
FILE *fp;
clrscr();
printf("How many records ?");
scanf("%d",&n);
fp=fopen("stud.dat","w");
for(i=0;i<n;i++)
{printf("Enter the student information : %d(sno,Name,M1,M2,M3):",i+1);
scanf("%d%s%d%d%d",&s.sno,s.name,&s.m1,&s.m2,&s.m3);
fwrite(&s,sizeof(s),1,fp);
}
fclose(fp);
fp=fopen("stdu.dat","r");
do
{printf("1-DISPLAY\n2.SEARCH\n3.EXIT\n YOUR OPTION: ");
scanf("%d",&open);
switch(open)
{
case 1:
printf("\n Student Records in the file \n");
```

```

display(fp);
break;
case 2:
printf("Read sno of the student to be searched :");
scanf("%d",&sno_key);
if(search(fp,sno_key)){
printf("success!! Record found in the file\n");
printf("%d\t%s\t%d\t%d\t%d\n", s.sno,s.name,s.m1,s.m2,s.m3);
}
else
printf("Failure!! Record %d not found\n",sno_key);
break;
case 3:
printf("Exit !! press key");
getch();
break;
default:
printf("Invalid option!!! Try again!!\n");
break;
}
}while(opn!=3);
fclose(fp);
}
Void display(FILE *fp)
{rewind(fp);
while(fread(&s,sizeof(s),1,fp))
printf("%d\t%s\t%d\t%d\t%d\n",s.sno,s.name,s.m1,s.m2,s.m3);
}
int search(FILE *fp,int sno_key)
{rewind(fp);
while(fread(&s,sizeof(s),1,fp))
If(s.sno==sno_key)
return 1;
return 0;
}

```

OUTPUT:

How many records?: 4

Enter the student information :1(sno,Name,m1,m2,m3): 1 Keerthana 100 99 98

Enter the student information :1(sno,Name,m1,m2,m3): 2 Nitheesh 97 98 96

Enter the student information :1(sno,Name,m1,m2,m3): 3 Rajasekhar 95 93 85

Enter the student information :1(sno,Name,m1,m2,m3): 4 Ramesh 94 99 100

1-DISPLAY

2-SEARCH

3-EXIT

YOUR OPTION 1

STUDENT RECORDS IN THE FILE

1	Keerthana	100	99	98
2	Nitheesh	97	98	96
3	Rajasekhar	95	93	85
4	Ramesh	94	99	100

1-DISPLAY

2-SEARCH

3-EXIT

YOUR OPTION 2

Read sno of the student to be searched: 1

Success!! Record found in the file

1	Keerthana	100	99	98
---	-----------	-----	----	----

RESULT:

Thus the Sequential file allocation method is implemented successfully.

AIM:

To Write a C Program to implement Indexed File Allocation method.

ALGORITHM:

Step1:Start.

Step2:Let n be the sizeof the buffer

Step 3:check if there are any producer

Step 4:If yes check whether the buffer isfull

Step 5:If no the producer item is stored in thebuffer

Step 6:If the buffer is full the producer has to wait

Step 7:Check there is any consumer. If yes check whether the buffer isempty

Step 8: If no the consumer consumes them from the buffer

Step 9:If the buffer isempty,the consumer has to wait.

Step10:Repeat checking for the producer and consumer till required

Step 11: Terminate the process.

PROGRAM:

```
#include<stdio.h>

main()
{
    int n,m[20],i,j,sb[20],s[
20],b[20][20],x;
    clrscr();

    printf("Enter no. of files:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter starting block and sizeof file%d:",i+1); scanf("%d
%d",&sb[i],&s[i]);

        printf("Enter blocks occupied by file%d:",i+1); scanf("%d",&m[i]);
        printf("enter blocks of file%d:",i+1); for(j=0;j<m[i];j++)
            scanf("%d",&b[i][j]);
    }
    printf("\nFile\tindex\tlength\n"); for(i=0;i<n;i++)
    {
```

```

        printf("%d\t%d\t%d\n",i+1,sb[i],m[i]);
    }
    printf("\nEnter file name:"); scanf("%d",&x);

    printf("file name is:%d\n",x); i=x-1;

    printf("Index is:%d",sb[i]); printf("Block occupied are:"); for(j=0;j<m[i];j++)
    printf("%3d",b[i][j]);

}

```

OUTPUT:

```

Enter no.of files:2
Enter starting block & size of file 1:2 5
Enter blocks occupied by file1:10
Enter blocks of file 1:3 2 5 4 6 7 2 6 4 7
Enter starting block and size of file 2:3 4
Enter blocks occupied by file 2:5
Enter blocks of file 2: 2 3 4 5 6
File          index      length
1             2         10
2             3         5
Enter file name:midname
File name is:12803
Index is 0
Block occupied

```

RESULT:

Thus the indexed file allocation method is implemented successfully

AIM:

To write a C program for random access file for processing the employee details.

ALGORITHM:

Step-1: Start the program.

Step-2: Get the number of records user want to store in the system.

Step-3: Using Standard Library function open the file to write the data into the file.

Step-4: Store the entered information in the system.

Step-5: Using do..While statement and switch case to create the options such as
1-DISPLAY, 2.SEARCH, 3.EXIT.

Step-6: Close the file using fclose() function.

Step-7: Process it and display the result.

Step-8: Stop the program.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
struct record
{
char empname[20];
int age;
float salary;
};
typedef struct record person
FILE *people;
void main()
{
person employee;
int I,n;
FILE *fp;
printf("How many records:");
scanf("%d",&n);
fp=fopen("PEOPLE.txt","w");
for(i=0;i<n;i++)
{
printf("Enter the employee information :%d(EmpName, Age,Salary):",i+1);
scanf("%s%d%f",employee.empname,&employee.age,& employee.salary);
fwrite(&employee,sizeof(employee),1,people);
}
fclose(fp);
int rec,result;
people=fopen("PEOPLE.txt","r");
printf("Which record do you want to read from file?);
```

```

scanf("%d",&rec);
while(rec>=0)
{
fseek(people,rec*sizeof(employee),SEEK_SET);
result=fread(&employee,sizeof(employee),1,people)
if(result==1)
{
printf("\n RECORD %d\n",rec);
printf("Given name:%s\n", employee.empname);
printf("Age:%d years\n",employee.age);
printf("Current salary:$ %8.2f\n\n",employee.salary);
}
else
printf( "\n RECORD %d not found !\n\n",rec);
printf("Which record do you want(0to3)"):
scanf("%d"<&rec);
}
fclose(people);
getch();
}

```

OUTPUT:

How many records?: 4

Enter the student information :1(sno,Name,m1,m2,m3): 1 Keerthana 100 99 98

Enter the student information :1(sno,Name,m1,m2,m3): 2 Nitheesh 97 98 96

Enter the student information :1(sno,Name,m1,m2,m3): 3 Rajasekhar 95 93 85

Enter the student information :1(sno,Name,m1,m2,m3): 4 Ramesh 94 99 100

1-DISPLAY

2-SEARCH

3-EXIT

YOUR OPTION 1

STUDENT RECORDS IN THE FILE

1 Keerthana 100 99 98

2 Nitheesh 97 98 96

3 Rajasekhar 95 93 85

4 Ramesh 94 99 100

1-DISPLAY

2-SEARCH

3-EXIT

YOUR OPTION 2

Read sno of the student to be searched: 1

Success!! Record found in the file

1 Keerthana 100 99 98

RESULT:

Thus the implementations of file allocation using linked access have been executed successfully.

Ex.No.15a Implementation of FCFS disk scheduling algorithms

Aim:

To implement FCFS the simplest of all the Disk Scheduling Algorithms. In FCFS, the Requests are addressed in the order they arrive in the disk queue.

ALGORITHM:

- Step 1: Start the program
- Step 2: Declare the necessary header files
- Step 3: Declare the variables which is needed to execute FCFS
- Step 4: Get the number of variables RQ[i], using for loop
- Step 5: Assign the logic of the FCFS disk scheduling
- Step 6: Execute till the process get completed
- Step 7: Set the initial head position
- Step 8: Print the results Total Head Moment
- Step 9: Stop the execution

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

int main()
{
    int RQ[100], i, n, TotalHeadMoment=0, initial;

    printf("Enter the number of Requests\n");

    scanf("%d", &n);

    printf("Enter the Requests sequence\n");

    for(i=0; i<n; i++)

        scanf("%d", &RQ[i]);

    printf("Enter initial head position\n");

    scanf("%d", &initial);

    for(i=0; i<n; i++)

    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial); initial=RQ[i];
    }

    printf("Total head moment is %d", TotalHeadMoment);

    return 0;
}
```

OUTPUT:

Enter the number of Request 8

Enter the Requests Sequence 95 180 34 119 11 123 62 64

Enter initial head position 50

Total head movement is 644

RESULT:

Thus the above Disk scheduling FCFS is implemented successfully

Ex.No.15b Implementation of SSTF disk scheduling Algorithm

AIM:

To implement Shortest seek time first(SSTF)algorithm, selects the disk I/O request which requires the least disk arm movement from its current position regardless of the direction. It reduces the total seek time as compared to FCFS

ALGORITHM:

- Step 1: Start the program
- Step 2: Declare the necessary header files
- Step 3: Declare the variables which is needed to execute SSTF
- Step 4: Get the number of variables RQ[i],using for loop
- Step 5: Assign the logic of the SSTF disk scheduling
- Step 6: Execute till the process get completed
- Step 7: Set the initial head position
- Step 8: Print the results TotalHeadMoment
- Step 9: Stop the execution

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int RQ[100],i,n,TotalHeadMoment=0,initial,count=0;

    printf("Enter the number of Requests\n");

    scanf("%d",&n);

    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++)
        scanf("%d",&RQ[i]);

    printf("Enter initial head position\n");
    scanf("%d",&initial);while(count!=n)
    {
        int min=1000,d,index;
        for(i=0;i<n;i++)
        {
            d=abs(RQ[i]-initial); if(min>d)
            {min=d; index=i;
            }
        }
        TotalHeadMoment=TotalHeadMoment+min; initial=RQ[index];
```

```
//1000is for max
//you can use any number RQ[index]=1000;
count++;
}
printf("Total head movement is%d", TotalHeadMoment); return 0;
}
```

OUTPUT:

Enter the number of Request 8

Enter Request Sequence

95 180 34 119 11 123 62 64

Enter initial head Position 50

Total head movement is 236

RESULT:

Thus the above Disk scheduling SSTF is implemented successfully.